UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

WorkSheet - 5

Student Name: Arnab Lala **UID: 25MCI10232**
Branch: M.C.A(A.l & M.L) **Section/Group: MAM-1 A**
Semester: SEM II **Date of Performance:24/02/2026**
Subject Name: TECHINCAL SKILLS Subject Code: 25CAP-652_25MAM_KAR-1(A)

AIM: To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic. **(Company Tags: Infosys, Wipro, TCS, Capgemini)**

S/W Requirement: Oracle Database Express Edition and pgAdmin

OBJECTIVES:

- **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
- **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
- **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
- **Exception Handling:** To handle cursor-related errors and performance considerations during large-scale data iteration.

Practical / Experiment Steps

```
/* ----------------------------------------------------------
   Create Staff Table
   ---------------------------------------------------------- */


CREATE TABLE Staff (
    staff_id SERIAL PRIMARY KEY,
    staff_name VARCHAR(60),
    base_pay NUMERIC(10,2),
    years_of_service INT,
    client_rating INT    -- Rating out of 5
);


INSERT INTO Staff (staff_name, base_pay, years_of_service, client_rating) VALUES
('Arjun', 50000, 3, 4),
('Meera', 82000, 9, 5),
('Dev', 45000, 2, 3),
```

```
('Sneha', 92000, 12, 5),
('Ishaan', 28000, 1, 2);
SELECT * FROM Staff;
```

	staff_id [PK] integer	staff_name character varying (60)	base_pay numeric (10,2)	years_of_service integer	client_rating integer
1	1	Arjun	69620.00	3	4
2	2	Meera	145049.80	9	5
3	3	Dev	57460.50	2	3
4	4	Sneha	177753.20	12	5
5	5	Ishaan	32659.20	1	2

```
/* ----------------------------------------------------------
   STEP 1: Cursor to Display Staff Details
```

Step 1: Implementing a Simple Forward-Only Cursor

```
   ---------------------------------------------------------- */
DO $$
DECLARE
    staff_rec RECORD;
    staff_cursor CURSOR FOR
        SELECT staff_id, staff_name, base_pay
        FROM Staff;
BEGIN
    OPEN staff_cursor;
    LOOP
        FETCH staff_cursor INTO staff_rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE
        'Staff ID: %, Name: %, Base Pay: %',
        staff_rec.staff_id,
        staff_rec.staff_name,
        staff_rec.base_pay;
    END LOOP;
    CLOSE staff_cursor;
END $$;
```

```
NOTICE:  Staff ID: 1, Name: Arjun, Base Pay: 69620.00
NOTICE:  Staff ID: 2, Name: Meera, Base Pay: 145049.80
NOTICE:  Staff ID: 3, Name: Dev, Base Pay: 57460.50
NOTICE:  Staff ID: 4, Name: Sneha, Base Pay: 177753.20
NOTICE:  Staff ID: 5, Name: Ishaan, Base Pay: 32659.20
DO
```

Step 2: Complex Row-by-Row Manipulation

```
DO $$
```

```
DECLARE
    staff_rec RECORD;
    staff_cursor CURSOR FOR
        SELECT staff_id, base_pay, years_of_service, client_rating
        FROM Staff;
    bonus_percent NUMERIC;
    updated_pay NUMERIC;
BEGIN
    OPEN staff_cursor;
    LOOP
        FETCH staff_cursor INTO staff_rec;
        EXIT WHEN NOT FOUND;
        /* Correct ROUND usage */
        bonus_percent :=
            ROUND(
                (staff_rec.years_of_service * 2) +
                (staff_rec.client_rating * 3),
2);
        updated_pay :=
            ROUND(
                staff_rec.base_pay +
                (staff_rec.base_pay * bonus_percent / 100),
            2);
        UPDATE Staff
        SET base_pay = updated_pay
        WHERE staff_id = staff_rec.staff_id;
        RAISE NOTICE
        'Staff ID: %, Bonus: %%%, Updated Pay: %',
        staff_rec.staff_id,
        bonus_percent,
        updated_pay;
    END LOOP;
    CLOSE staff_cursor;
END $$;
```

```
NOTICE:  Staff ID: 1, Bonus: %18.00, Updated Pay: 96938.89
NOTICE:  Staff ID: 2, Bonus: %33.00, Updated Pay: 256578.59
NOTICE:  Staff ID: 3, Bonus: %13.00, Updated Pay: 73371.32
NOTICE:  Staff ID: 4, Bonus: %39.00, Updated Pay: 343436.96
NOTICE:  Staff ID: 5, Bonus: %8.00, Updated Pay: 38093.70
```

STEP 3 — Exception Handling (Different Condition)

```
/* ------------------------------------------------------------
   STEP 3: Exception Handling Example
   ------------------------------------------------------------ */
DO $$
DECLARE
    staff_rec RECORD;
    staff_cursor CURSOR FOR
        SELECT staff_id, client_rating
        FROM Staff;
BEGIN
    OPEN staff_cursor;
    LOOP
        FETCH staff_cursor INTO staff_rec;
        EXIT WHEN NOT FOUND;
        -- Raise exception if rating is extremely poor
        IF staff_rec.client_rating < 2 THEN
            RAISE EXCEPTION
            'Unacceptable performance for Staff ID: %',
            staff_rec.staff_id;
        END IF;
        RAISE NOTICE
        'Checked Staff ID: %, Rating: %',
        staff_rec.staff_id,
        staff_rec.client_rating;
    END LOOP;
    CLOSE staff_cursor;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error handled successfully.';
        RAISE NOTICE 'Reason: %', SQLERRM;
```

```
    RAISE NOTICE 'Process terminated safely.';
END $$;
```

```
NOTICE:  Checked Staff ID: 1, Rating: 4
NOTICE:  Checked Staff ID: 2, Rating: 5
NOTICE:  Checked Staff ID: 3, Rating: 3
NOTICE:  Checked Staff ID: 4, Rating: 5
NOTICE:  Checked Staff ID: 5, Rating: 2
```

Outcomes:

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.